Text-to-speech Implementation process

Website link

Android Real-Time TTS: Streaming Text-to-Speech Tutorial [2025]

PUBLISHED: NOVEMBER 27, 2025 · 4 MIN READ

Engineering Text-to-Speech

Streaming Text-to-Speech (TTS) enables Android apps to generate and play audio incrementally as text arrives, which is essential for real-time voice interfaces, accessibility features, and conversational assistants.

A major limitation on Android is that the native TextToSpeech API **cannot produce streaming or token-by-token audio**. It requires complete text before synthesis, making it unsuitable for real-time applications like handling partial LLM outputs.

Cloud-based TTS services such as Amazon Polly, Azure TTS, ElevenLabs, and OpenAI TTS introduce additional issues: network latency, dependency on connectivity, and privacy concerns. Even the fastest cloud engines can add hundreds to thousands of milliseconds of delay, whereas **on-device TTS begins synthesizing immediately and delivers audio** 6.5x faster than the closest competitor (ElevenLabs).

The solution is **on-device, streaming speech synthesis** with Orca Streaming Text-to-Speech. This tutorial demonstrates **how to build real-time speech generation on Android** using `Orca` for voice synthesis and Android's AudioTrack API for **PCM audio streaming**. The approach works with any streaming text source—including live LLM output (ChatGPT, Claude, or picoLLM On-device LLM Inference) or dynamically generated content.

Key benefits for enterprise developers:

- **Low-latency streaming**: Audio plays as text arrives; 130 ms first-word latency
- **On-device processing**: Runs in environments with unreliable network connectivity
- **Flexible text sources**: Works with LLMs, user input, or any streaming text source

How to Build Streaming TTS on Android

Prerequisites

Before you begin, make sure you have the following:

- Android Studio
- Android device or emulator (Android 7.0 API 24 or higher)
- USB debugging enabled on your Android device
- Picovoice Account and AccessKey

1. Project Setup

This tutorial demonstrates a project built with Kotlin with Jetpack Compose, targeting **Android 15 (API level 35)** with a minimum supported version of **Android 7.0 (API level 24)**.

Add Internet Permission

Include this in your `AndroidManifest.xml`:

```
<uses-permission android:name="android.permission.INTERNET" />
```

> `Orca Streaming Text-to-Speech` requires internet connectivity only for authenticating your `AccessKey`. All speech synthesis runs **entirely on-device**.

2. Add Orca Library and Model File

2a. Add Orca Library via Maven Central

In `app/build.gradle.kts`, add orca-android to your `dependencies`:

```
dependencies {
    implementation(libs.orca.android)
}
```

Then in `gradle/libs.versions.toml` (replace `{LATEST_VERSION}`, e.g. `1.2.0`):

```
[versions]
orcaAndroid = "{LATEST_VERSION}"

[libraries]
orca-android = { module = "ai.picovoice:orca-android", version.ref = "orcaAndroid" }
```

2. Add Orca Library and Model File

2a. Add Orca Library via Maven Central

In `app/build.gradle.kts`, add orca-android to your `dependencies`:

```
dependencies {
    implementation(libs.orca.android)
}
```

Then in `gradle/libs.versions.toml` (replace `{LATEST_VERSION}`, e.g. `1.2.0`):

Copy

```
[versions]
orcaAndroid = "{LATEST_VERSION}"

[libraries]
orca-android = { module = "ai.picovoice:orca-android", version.ref = "orcaAndroid" }
```

Execute a Gradle sync.

2b. Add Orca Model File

`Orca` uses **model files** (`.pv`) for different languages and voices.

1. Download your desired model from the Orca GitHub repository. The filename indicates the **language** and **gender** of the speaker.
2. Place the model in your Android project under: `{ANDROID_APP}/src/main/assets`

4. Playing Synthesized Speech with AudioTrack

Once you have PCM audio chunks in a queue, you can play them using AudioTrack, which streams raw PCM audio to the device's speakers.

4a. Configure AudioTrack

`Orca` outputs **mono, 16-bit PCM, with a sample rate of 22050 Hz**, which is a common format for speech synthesis. Using `AudioTrack` in streaming mode allows you to play audio chunks incrementally, keeping latency low.

```
val sampleRate = orca?.sampleRate ?: 22050
val minBuffer = AudioTrack.getMinBufferSize(
    sampleRate,
    AudioFormat.CHANNEL_OUT_MONO,
    AudioFormat.ENCODING_PCM_16BIT
)

val audioTrack = AudioTrack.Builder()
    .setAudioFormat(
        AudioFormat.Builder()
            .setEncoding(AudioFormat.ENCODING_PCM_16BIT)
            .setSampleRate(sampleRate)
            .setChannelMask(AudioFormat.CHANNEL_OUT_MONO)
            .build()
    )
    .setBufferSizeInBytes(minBuffer)
    .setTransferMode(AudioTrack.MODE_STREAM)
    .build()
```

Explanation of key settings:

- `ENCODING_PCM_16BIT`: Matches `Orca`'s 16-bit PCM output.
- `CHANNEL_OUT_MONO`: Single-channel audio for voice playback; matches `Orca`'s mono PCM output.
- `MODE_STREAM`: Enables incremental writing of audio data as it's synthesized, instead of buffering everything first.